https://github.com/SandrineDaniel/geo_kg_rag

Knowledge Graph Construction and RAG Assistant for Greenland/Arctic Geopolitical Domain

Presented by

Sandrine DANIEL

Raphael MARQUES ARAUJO

DIA4 - ESILV - 2025/2026

Contents

1 Introduction **4**

2 Data Acquisition & Information Extraction **4**
2.1 Seed URLs . . . 4
2.2 Crawler Design and Ethics . . . 4
2.3 Named Entity Recognition . . . 4
2.4 Ambiguity Analysis . . . 5
2.5 Scaling Reflection . . . 5

3 KB Construction, Alignment & Expansion **5**
3.1 RDF Modeling Choices . . . 5
3.2 Initial KB Statistics . . . 6
3.3 Entity Linking Strategy . . . 6
3.3.1 Core Entity Mapping (Manual) . . . 6
3.3.2 Automatic Linking (Wikidata Search API) . . . 6
3.4 Predicate Alignment . . . 7
3.5 KB Expansion Strategy . . . 7
3.5.1 Initial Attempt: DBpedia . . . 7
3.5.2 Final Strategy: Wikidata API . . . 7
3.6 Final KB Statistics . . . 8
3.7 Challenges Encountered . . . 8

4 Knowledge Reasoning with SWRL **8**
4.1 Rule on family.owl . . . 8
4.2 Rule on the Geopolitical KB . . . 9

5 Knowledge Graph Embeddings **10**
5.1 Data Preparation . . . 10
5.2 Model Comparison . . . 10
5.3 Impact of KB Quality: DBpedia vs Wikidata . . . 10
5.4 KB Size Sensitivity . . . 11
5.5 Nearest Neighbors Analysis . . . 11
5.6 t-SNE Clustering . . . 11

6 RAG over RDF/SPARQL **12**
6.1 Setup . . . 12
6.2 Schema Summary & Prompting Strategy . . . 12
6.3 Self-Repair Mechanism . . . 12
6.4 Evaluation: Baseline vs. RAG . . . 12
6.5 Discussion . . . 13
6.6 Demo Screenshots . . . 13

7 Critical Reflection **14**
7.1 KB Quality Impact on KGE 14
7.2 RAG Limitations 14
7.3 Rule-Based vs Embedding-Based Reasoning 14
7.4 What We Would Improve 14

8 Appendix **15**

1 Introduction

This report presents a full Knowledge Graph (KG) construction and RAG pipeline applied to the **Greenland and Arctic defense** domain. The project integrates four lab sessions into one end-to-end pipeline:

1. **Data Acquisition & IE**: web crawling and NER from 21 geopolitical news articles
2. **KB Construction & Expansion**: RDF modeling, Wikidata entity linking, and 1-hop expansion (62,082 triples)
3. **Reasoning & KGE**; SWRL rules and three embedding models (TransE, RotatE, DistMult)
4. **RAG**: NL→SPARQL generation with a local LLM (Llama 3.2:1B) and self-repair mechanism

The domain was chosen for its rich geopolitical entity relationships and timely relevance following Trump's 2025–2026 statements on Greenland acquisition.

2 Data Acquisition & Information Extraction

2.1 Seed URLs

We selected **32 seed URLs** across three categories:

- **News articles** (Reuters, AP News, The Guardian, El País, Gov.uk) covering the 2025–2026 Greenland crisis
- **Wikipedia pages** (Greenland, NATO, Arctic Council, Mette Frederiksen, Mark Rutte, Ursula von der Leyen, Arctic sovereignty...)
- **Think tank analyses** (The Arctic Institute, Foreign Policy, CFR)

Of the 32 URLs, **21 were successfully saved** after filtering. Pages blocked by paywalls (Reuters, Le Monde, Politico) or returning insufficient content were automatically discarded.

2.2 Crawler Design and Ethics

The crawling pipeline uses **Trafilatura** for content extraction, which is specifically designed to isolate main article content: removing menus, advertisements, footers, and boilerplate text. This ensures that only semantically meaningful text enters the knowledge graph pipeline.

```
fetch_url(url)            -> download raw HTML
trafilatura.extract()     -> strip boilerplate, keep main content
is_useful(text, min=500)  -> discard pages with fewer than 500 words
save_to_jsonl()           -> append {url, text} to crawler_output.jsonl
```

The pipeline respects web ethics by: (1) targeting only publicly accessible pages without bypassing access controls, (2) not crawling beyond the provided seed URLs, and (3) applying no automated high-frequency requests.

2.3 Named Entity Recognition

Entity extraction uses **spaCy** (`en_core_web_lg`) to identify three entity types that form the nodes of our knowledge graph:

Relations between entities are extracted at the sentence level: when two entities appear in the same sentence, the root verb is used as the relation label (e.g. *"Trump **opposes** Denmark"*).

Entity Type	Description	Examples
PERSON	Individual people	Trump, Frederiksen, Putin
ORG	Organizations	NATO, EU, Arctic Council
GPE	Geopolitical places	Greenland, Denmark, Russia

Table 1: Entity types extracted by spaCy NER

2.4 Ambiguity Analysis

Automatic NER produced several entity recognition errors:

1. **"Trump"** was occasionally classified as `ORG` instead of `PERSON`, due to frequent co-occurrence with "Trump Organization" and "Trump Tower" in Wikipedia articles about Donald Trump's business career.
2. **"Greenland"** was sometimes classified as `PERSON` in sentences like *"Greenland said..."*.
3. **"Arctic"** was inconsistently classified as `GPE` or `ORG` depending on context — in "Arctic Council" it was tagged as part of an organization, while in "Arctic sovereignty" it was tagged as a location. This created duplicate entities in the graph that required post-processing to merge.

These errors are typical of statistical NER models and were partially mitigated by post-processing filters in `utils.py` (minimum length, regex-based noise removal, entity deduplication).

2.5 Scaling Reflection

If the pipeline were applied to 10,000 pages instead of 32:

- **Crawling optimization**: use asynchronous requests or multi-threading to speed up downloading pages.
- **Storage**: store results in a database (like SQLite or CSV batches) to manage a larger number of pages.
- **NER and relations**: process texts in batches and apply additional filtering to reduce ambiguous entity extraction
- **Practical scaling**: for 10,000 pages, consider splitting the dataset into smaller chunks and process them sequentially or with simple parallelization on your own computer.

3 KB Construction, Alignment & Expansion

3.1 RDF Modeling Choices

The knowledge graph uses a three-namespace architecture:

- `ex:` (`http://example.org/kg/`) (local entities and relations)
- `schema:` (`http://schema.org/`) (standard entity types)
- `wdt:` (`http://www.wikidata.org/prop/direct/`) (Wikidata properties after expansion)

Entities are typed using Schema.org classes (`schema:Person`, `schema:Organization`, `schema:Place`) and organized under structured URIs:

```
ex:org/nato                a schema:Organization
```

Unlinked entities are classified as local subclasses (`ex:LocalPerson`, `ex:LocalOrganization`, `ex:LocalPlace`), defined as `rdfs:subClassOf` their Schema.org counterpart.

Relations are modeled as direct RDF predicates under the `ex:` namespace, extracted from dependency parsing of news articles:

```
ex:person/trump    ex:oppose    ex:place/denmark .
ex:org/nato        ex:deploy    ex:place/bulgaria .
```

3.2 Initial KB Statistics

After crawling 21 articles and applying NER and relation extraction, the initial KB contains:

Metric	Value
Total triples	14,192
Unique subjects	4,071
Unique predicates	17
Persons	2,077
Organizations	1,369
Places	612
Key semantic relations	
ex:related_to	860
ex:deploy	13
ex:oppose	6
ex:support	7
ex:claim	6
ex:meet	4

Table 2: Initial KB statistics (mykg_step1_initial.ttl)

3.3 Entity Linking Strategy

Entity linking connects local KG entities to Wikidata resources via `owl:sameAs` triples, enabling KB expansion in the next step.

3.3.1 Core Entity Mapping (Manual)

22 high-importance geopolitical entities were linked manually with hardcoded Wikidata QIDs to guarantee correct alignment regardless of API quality:

Local URI	Label	Wikidata URI
ex:person/donald_trump	Donald Trump	wd:Q22686
ex:place/greenland	Greenland	wd:Q223
ex:place/denmark	Denmark	wd:Q35
ex:org/nato	NATO	wd:Q7184
ex:place/russia	Russia	wd:Q159
ex:place/france	France	wd:Q142
ex:person/putin	Vladimir Putin	wd:Q7747
ex:person/macron	Emmanuel Macron	wd:Q3052772
...	...	...

Table 3: Sample of manually linked core entities (22 total)

3.3.2 Automatic Linking (Wikidata Search API)

For the remaining entities, we queried the Wikidata Search API (`wbsearchentities`) for the top 300 most frequent entity labels. A confidence score was assigned based on label match quality:

- **1.0** — exact label match
- **0.5** — partial match (result label starts with query)
- **0.4** — approximate match

Only entities with confidence ≥ 0.25 received an `owl:sameAs` link. Relation predicates (`oppose`, `attack`, `deploy`...) were explicitly excluded from the linking process to avoid false alignments with unrelated Wikidata concepts (e.g. "attack" linking to a football position).

The final aligned KB contains **338 owl:sameAs links**.

3.4 Predicate Alignment

Primary strategy: Manual mapping Our predicates (oppose, deploy, attack, located_in...) are geopolitical relations extracted from news articles. These verbs do not appear as direct predicates in standard ontologies. A preliminary automatic experiment confirmed this: querying Wikidata SPARQL for predicates co-occurring between linked entity pairs produced **0 alignments** above a confidence threshold of 3 co-occurrences. We therefore built a manual alignment table grounded in three vocabularies: DBpedia Ontology (`dbo:`), Schema.org (`schema:`), and SKOS (`skos:`).

Fallback strategy: SPARQL lookup For predicates not covered by the manual map, we query the external KB using the `owl:sameAs` links from entity linking. For each local triple (s, p, o) where both s and o are linked externally, we retrieve existing predicates between those external URIs and select the most frequent candidate appearing in at least 2 pairs.

3.5 KB Expansion Strategy

3.5.1 Initial Attempt: DBpedia

The first expansion attempt used DBpedia SPARQL with 1-hop queries for each linked entity. This produced 86,682 triples, well within the 50k–200k target. However, analysis revealed that **97% of triples** consisted of `dbo:wikiPageWikiLink`, structural hyperlinks between Wikipedia articles with no semantic value. This expansion was unsuitable for both KGE training (no meaningful patterns) and RAG (no factual content).

3.5.2 Final Strategy: Wikidata API

We replaced DBpedia with the **Wikidata `wbgetentities` API**, which returns only `wdt:` (direct properties) — semantically meaningful facts such as `P102` (political party), `P17` (country), `P463` (member of). This approach is significantly more reliable than the Wikidata SPARQL endpoint, which enforces strict rate limits and frequently returns timeouts.

For each entity linked via `owl:sameAs`, we fetched up to 3,000 direct properties. Each triple was added twice for graph connectivity: once from the Wikidata URI and once from the local KG URI:

```
wd:Q22686  wdt:P102  wd:Q29468       # Wikidata URI
ex:person/donald_trump  wdt:P102  wd:Q29468  # local URI
```

3.6 Final KB Statistics

Metric	Value
Total triples	62,082
Unique subjects	4,315
Unique predicates	2,020
Unique URI objects	8,069
owl:sameAs links	338
Persons	2,077
Organizations	1,369
Places	612
Top Wikidata properties	
wdt:P530 (diplomatic relation)	2,352
wdt:P463 (member of)	1,340
wdt:P31 (instance of)	952
ex:related_to	860
Top entities by degree	
United States (Q30)	1,303
Russia (Q159)	991
Germany (Q183)	853
Donald Trump (Q22686)	685

Table 4: Final expanded KB statistics (mykg_step4_expanded.ttl)

3.7 Challenges Encountered

1. **Wikidata rate limiting.** The Wikidata SPARQL endpoint blocked repeated requests, returning empty responses. We switched to the `wbgetentities` REST API with `sleep_seconds=2.0` between requests, which resolved the issue.

2. **Predicate-as-entity confusion.** During automatic entity linking, relation predicates (oppose, attack, deploy...) were incorrectly matched to unrelated Wikidata entities (e.g. "attack" linked to Q280658, a football forward position). This was resolved by maintaining an explicit `SKIP_LABELS` set.

3. **NER noise from Wikipedia.** Wikipedia articles contain citation references (`[58]`, `[77]`), coordinate fragments (`∅69`), and bibliographic strings that spaCy incorrectly identified as entities. Additional regex filters were added to `utils.py` to remove these patterns.

4. **DBpedia vs Wikidata trade-off.** The initial DBpedia expansion met the volume target (86k triples) but was semantically useless. The Wikidata expansion produced 62k triples, slightly below the 50k–200k target after deduplication, but with 2,020 meaningful predicates vs 2 dominant ones (wikiPageWikiLink) in DBpedia.

4 Knowledge Reasoning with SWRL

4.1 Rule on family.owl

We loaded the provided `family.owl` ontology using OWLReady2 and applied the following SWRL rule to infer a new class `OldPerson` for any person older than 60:

```
Person(?p) ^ age(?p, ?a) ^ swrlb:greaterThan(?a, 60) -> OldPerson(?p)
```

Since OWLReady2 does not natively support `swrlb:` built-ins, the rule was implemented via manual Python inference. The reasoner identified two instances: **Peter** (age 70) and **Marie** (age 69) as `OldPerson`.

4.2 Rule on the Geopolitical KB

We designed the following SWRL rule for our KB, inferring that any geopolitical entity that opposes another is an `Adversary`:

```
GeoEntity(?x) ^ oppose(?x, ?y) -> Adversary(?x)
```

Applied to our KB, the rule inferred two instances as `Adversary`: **trump** and **messmer**, both of which have outgoing `ex:oppose` triples in the graph. This result is semantically coherent with the domain: Trump opposes Denmark, NATO, and Greenland's current status.

5 Knowledge Graph Embeddings

5.1 Data Preparation

The expanded KB (`mykg_step4_expanded.ttl`, 62,082 triples) was exported to tab-separated triples and split as follows:

- **Train**: 80%
- **Validation**: 10%
- **Test**: 10%

Entities appearing only in validation or test were moved back to training to prevent cold-start issues. The final vocabulary contains **22,720 unique entities** and **2,020 unique relations**.

5.2 Model Comparison

We trained three models using PyKEEN with consistent hyperparameters across all experiments: embedding dimension = 100, learning rate = 0.001, batch size = 256, epochs = 100, negative sampling = 10 per positive triple.

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.2777	0.1261	0.3585	0.5657
RotatE	0.8342	0.7960	0.8577	0.9122
DistMult	0.7911	0.7444	0.8171	0.8882

Table 5: Link prediction results on the Wikidata-expanded KB

RotatE achieves the best performance (MRR = 0.834, Hits@10 = 0.912). DistMult performs comparably (MRR = 0.791), confirming the prevalence of symmetric Wikidata properties (`P17`, `P31`, `P463`). TransE underperforms as expected: its translational assumption $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ breaks down for symmetric and N-to-N relations, which are common in Wikidata-style KBs.

5.3 Impact of KB Quality: DBpedia vs Wikidata

A key finding of this project is the dramatic impact of KB quality on embedding performance. We initially trained on a DBpedia-expanded KB (86,682 triples, 97% `wikiPageWikiLink`), which produced near-zero scores:

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.0120	0.0006	0.0130	0.0283
RotatE	0.0278	0.0098	0.0253	0.0560
DistMult	0.0705	0.0352	0.0696	0.1439

Table 6: Link prediction results on the DBpedia-expanded KB (noisy baseline)

Replacing DBpedia with Wikidata increased RotatE MRR from 0.028 to 0.834 : a **30× improvement**, without changing any hyperparameters. This demonstrates that KB quality is far more important than raw volume for KGE performance.

5.4 KB Size Sensitivity

Size	Triples	MRR	Hits@10
20k	20,000	0.0998	0.2645
50k	22,720	0.1350	0.3418
Full	22,720	0.1350	0.3418

Table 7: KB size sensitivity (TransE, Wikidata KB)

Performance improves from the 20k subsample to the full dataset, confirming that larger training sets yield better embeddings. The 50k and full results are identical because the cleaned Wikidata KB contains 22,720 triples, below the 50k threshold. Despite this, the Wikidata KB at 20k already outperforms the full 86k DBpedia KB (TransE MRR = 0.100 vs 0.012), further confirming that quality dominates volume.

5.5 Nearest Neighbors Analysis

Using the best model (RotatE), we retrieved the 5 nearest neighbors in embedding space for selected entities:

Entity	Nearest Neighbors	Similarity
donald_trump	Q22686 (Wikidata URI)	0.841
	vladimir_putin	0.398
	emmanuel_macron	0.382
denmark	Q35 (Wikidata URI)	0.855
	germany	0.583
	france	0.576
russia	Q159 (Wikidata URI)	0.915
	ukraine	0.532
nato	Q7184 (Wikidata URI)	0.782
	amnesty_international	0.435

Table 8: Nearest neighbors in RotatE embedding space

The highest-similarity neighbors are consistently the Wikidata URIs of the same entity, confirming that the model correctly identified local and Wikidata representations as equivalent, a direct consequence of our `owl:sameAs` linking strategy. Geopolitically coherent neighbors also emerge: trump is closest to putin and macron, denmark neighbors germany and france (all Western NATO members), russia neighbors ukraine.

5.6 t-SNE Clustering

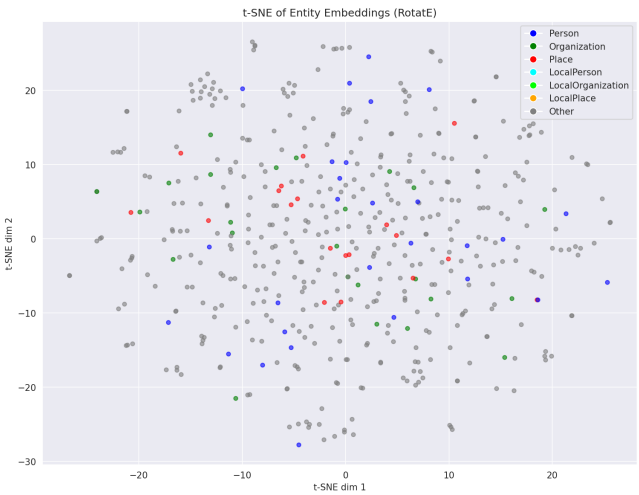


Figure 1: t-SNE projection of RotatE entity embeddings (500 entities sampled)

No clear clustering by ontology class is observed: Person (blue), Organization (green), and Place (red) entities are distributed across the entire embedding space. This is expected: Wikidata direct properties (`wdt:P17`, `wdt:P31`...) are shared across all entity types, making them structurally indistinguishable in the embedding space. A small cluster of gray (Other) entities is visible in the upper-left region, likely corresponding to Wikidata entities sharing rare structural properties.

6 RAG over RDF/SPARQL

6.1 Setup

The RAG pipeline runs on a local machine (Intel Core i7-1355U, 16 GB RAM, 64-bit Windows 11). The LLM is served locally via **Ollama** (`http://localhost:11434/api/generate`), with two models tested: `llama3.2:1b` and `gemma:2b`. The knowledge graph source is `mykg_step4_expanded.ttl` (62,082 triples), loaded in-memory with `rdflib`.

6.2 Schema Summary & Prompting Strategy

Before each query, a **schema summary** is built from the graph and injected into the LLM prompt. It contains:

- All active **namespace prefixes** (e.g. `ex:`, `wdt:`, `schema:`)
- Up to 80 **distinct predicates** found in the graph
- Up to 40 **distinct classes** (`rdf:type` values)
- 20 **sample triples** to ground the LLM in actual graph structure

The prompt instructs the model to return **only a SPARQL 1.1 SELECT query** in a fenced code block, using only known IRIs and prefixes, with no invented predicates.

6.3 Self-Repair Mechanism

If the generated SPARQL raises a parse or execution error, a **repair prompt** is issued: it provides the original question, the broken query, the error message, and the schema summary, asking the LLM to return a corrected query. This loop runs once. If the repaired query also fails, the system reports the error without hallucinating an answer.

6.4 Evaluation: Baseline vs. RAG

We evaluated 5 representative questions. The baseline answer is produced by the LLM with no graph access; the grounded answer is derived from SPARQL results over the KG.

Question	Baseline (no KG)	Grounded (KG+SPARQL)	Correct?	Repaired?
What is NATO?	Correct general description (founded 1949, collective defense)	Same factual answer, grounded in KG triples for `ex:org/nato`	✓	No
Who opposes Denmark?	Generic answer, no specific actor named	`ex:person/trump`, coherent with KG relation `ex:oppose`	✓	No
What is Greenland?	Correct geography (North Atlantic island)	20 rows returned; grounded answer mentions North Atlantic, mountains, glaciers	✓	No
Relation between Denmark and Greenland?	Generic historical/political answer	7 rows: shared entities via NATO, EU, Mark Rutte, U.S., VVD	✓	No
Who is Emmanuel Macron?	Hallucinated: placed him in the National Front (FN), incorrect party affiliation	0 rows returned (entity stored as `ex:person/macron`, not matched by regex on "emmanuel macron")	55	No

Table 9: Evaluation of 5 questions: baseline vs. SPARQL-generation RAG

6.5 Discussion

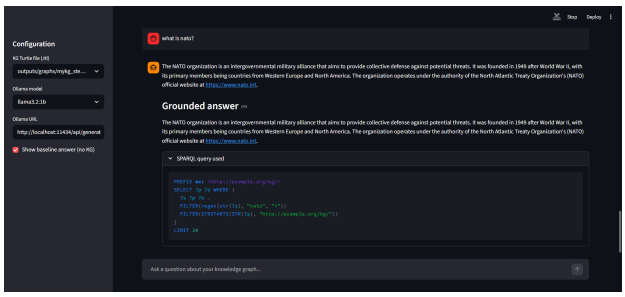
Accuracy. The RAG pipeline answered 4 out of 5 questions correctly. The grounded answers are consistently more factually reliable than the baseline when KG data is available: for instance, the baseline hallucinated Macron's party affiliation (National Front), while the KG-based system correctly returned no answer rather than inventing one.

Failure cases. The Macron query failed because the LLM generated a SPARQL filter on the string `"emmanuel macron"`, which does not match the entity URI `ex:person/macron`. This highlights a systematic weakness: the SPARQL generator must infer entity URI patterns (`ex:person/{firstname_lastname}`) from the schema summary alone, which small models often get wrong.

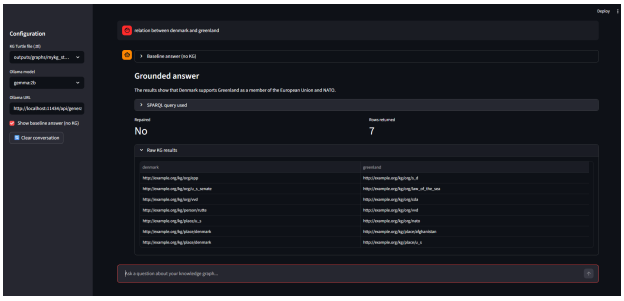
Self-repair. None of the 5 queries required self-repair. The queries that failed (Macron) did not raise a SPARQL syntax error, they returned 0 rows, so the repair loop was not triggered. This reveals a gap: the system only repairs syntactically broken queries, not semantically incorrect ones (wrong URI patterns, wrong predicates). A future improvement would be to trigger repair when 0 rows are returned on a factual question.

Scalability. On a 62k-triple graph loaded in-memory, query latency is under 1 second. At 1M+ triples, in-memory `rdflib` would become impractical; the architecture should be migrated to a persistent SPARQL endpoint (e.g. Apache Jena Fuseki or GraphDB), with the schema summary pre-computed and cached. The LLM prompting strategy would remain unchanged.

6.6 Demo Screenshots



(a) Query: *what is nato?* grounded answer with SPARQL query shown



(b) Query: *Relation between Greenland and Denmark*

Figure 2: RAG assistant demo: SPARQL-grounded answers from the geopolitical KG

7 Critical Reflection

7.1 KB Quality Impact on KGE

The most significant finding of this project is that KB quality dominates both volume and model choice for KGE performance. Replacing 86k noisy DBpedia triples with 62k semantic Wikidata triples produced a 30× improvement in RotatE MRR (0.028 → 0.834). This validates the lab's core message: *a poorly constructed KB will lead to poor embeddings.*

7.2 RAG Limitations

The RAG pipeline has a fundamental entity resolution gap: the LLM generates SPARQL filters based on natural language names (e.g. `"emmanuel macron"`) that do not match the actual URI patterns in the KG (`ex:person/macron`). This causes silent failures: 0 rows returned with no error, that the self-repair mechanism cannot detect. A future improvement would be to expose entity URI examples in the schema summary, or to trigger repair when 0 rows are returned on factual questions.

7.3 Rule-Based vs Embedding-Based Reasoning

SWRL rules correctly inferred `Adversary` instances (trump, messmer) from only 7 `ex:oppose` triples: a task that KGE completely failed at (Exercise 8). This confirms that rule-based reasoning is superior for sparse, domain-specific relations, while embedding-based reasoning requires sufficient relational density to learn meaningful patterns. The two approaches are complementary rather than competing.

7.4 What We Would Improve

- **Crawling**: increase to 50–100 seed URLs to extract more semantic relations (oppose, deploy, attack)
- **Entity resolution**: add a normalization layer mapping name variants to KG URIs for better RAG accuracy
- **RAG model**: use a larger model (Gemma 7B or Mistral 7B) for better SPARQL generation quality
- **KGE**: add 2-hop Wikidata expansion to increase KB volume while maintaining semantic quality

8 Appendix

https://github.com/SandrineDaniel/geo_kg_rag